

Генеративные модели

Лекция 5: *GAN*

Идея нормализующих потоков

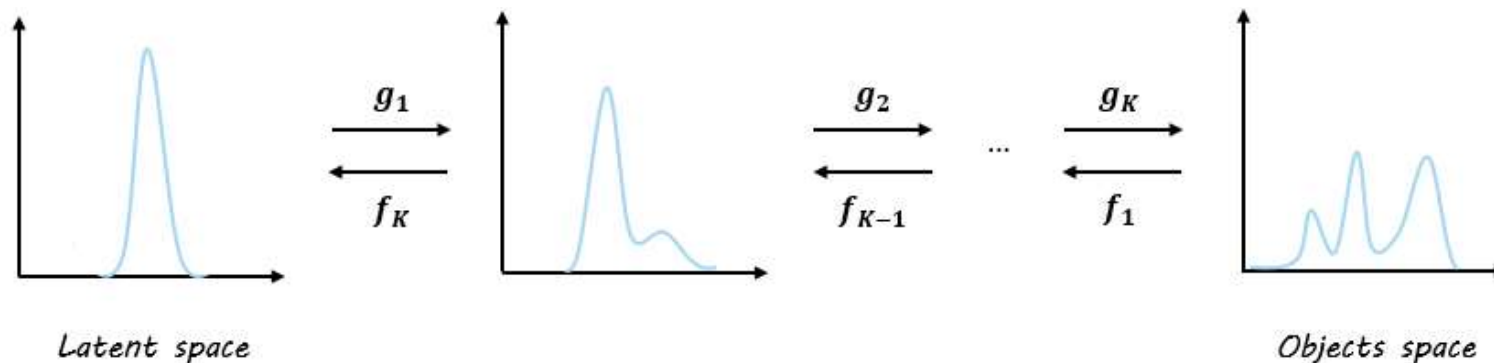
Особенность потоков – наличие двух направлений:

- **Нормализующее направление (для обучения)** $x \rightarrow z$:

Берём реальный объект x и с помощью нормализующей функции f превращаем его в простой z

- **Генеративное направление** $z \rightarrow x$:

Берём простой объект z и с помощью генеративной функции $g = f^{-1}$ создаём объект x



Теорема о замене переменных

Наша цель – найти распределение $p(\mathbf{x})$

Связать неизвестную плотность $p(\mathbf{x})$ с известной плотностью простого распределения $p(\mathbf{z})$ нам позволяет **формула замены переменных**:

$$p(\mathbf{x}) = p(\mathbf{z}) |\det(J_f)| = p(\mathbf{z}) \left| \det \left(\frac{\partial \mathbf{z}}{\partial \mathbf{x}} \right) \right| = p(\mathbf{f}(\mathbf{x})) \left| \det \left(\frac{\partial \mathbf{f}(\mathbf{x})}{\partial \mathbf{x}} \right) \right|$$

Здесь $J_f(\mathbf{x})$ – матрица Якоби:

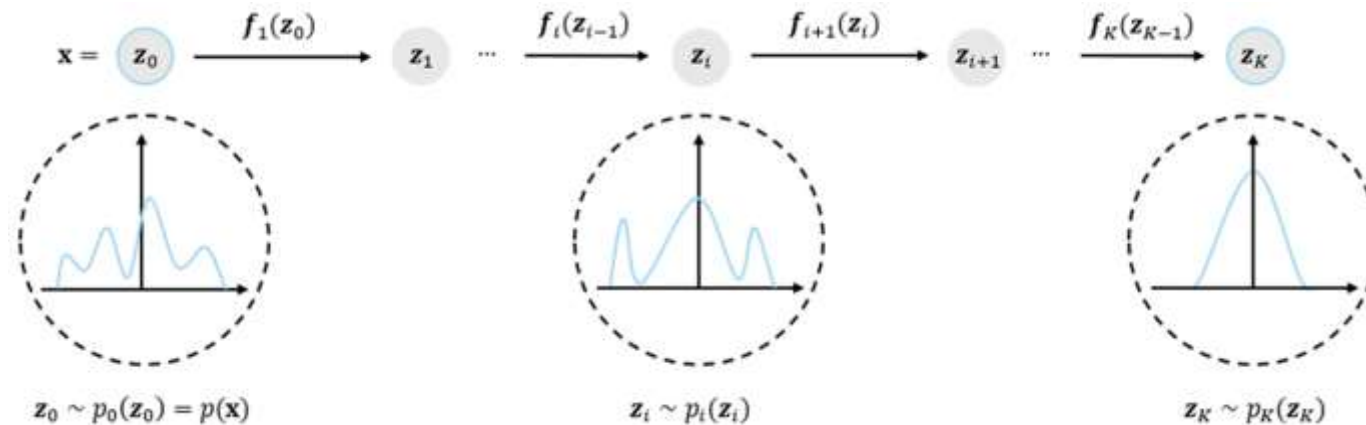
$$J_f(\mathbf{x}) = \begin{pmatrix} \frac{\partial f_1}{\partial x_1} & \dots & \frac{\partial f_1}{\partial x_D} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_D}{\partial x_1} & \dots & \frac{\partial f_D}{\partial x_D} \end{pmatrix}$$

Композиция преобразований

- Одной функции f недостаточно, чтобы из $\mathcal{N}(0,1)$ получить сложное распределение наших данных
- Потoki используют последовательность из K простых и обратимых преобразований:

$$\mathbf{x} = \mathbf{z}_0 \xrightarrow{f_1} \mathbf{z}_1 \xrightarrow{f_2} \dots \xrightarrow{f_K} \mathbf{z}_K$$

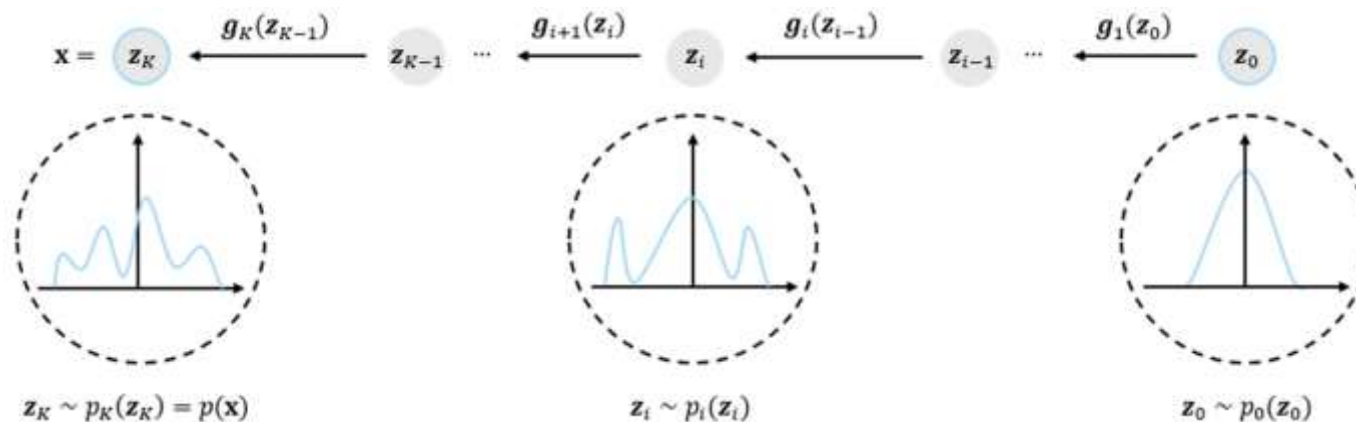
- Во время обучения мы используем нормализующее направление $\mathbf{x} \rightarrow \mathbf{z}$:



Композиция преобразований

- Генеративное направление $\mathbf{z} \rightarrow \mathbf{x}$ используется для создания новых объектов:

$$\mathbf{z}_0 \xrightarrow{g_1} \mathbf{z}_1 \xrightarrow{g_2} \dots \xrightarrow{g_K} \mathbf{z}_K = \mathbf{x}$$



Композиция преобразований

Нужно найти определитель якобиана всей композиции

Два математических факта:

- Матрица Якоби для композиций функций равна их произведению:

$$\frac{\partial \mathbf{f}}{\partial \mathbf{x}} = \frac{\partial \mathbf{f}_K}{\partial \mathbf{z}_{K-1}} \cdot \frac{\partial \mathbf{f}_{K-1}}{\partial \mathbf{z}_{K-2}} \cdot \dots \cdot \frac{\partial \mathbf{f}_1}{\partial \mathbf{z}_0}$$

- Определитель произведения матриц равен произведению определителей:

$$\det(\mathbf{J}_f) = \det(\mathbf{J}_{f_K}) \cdot \det(\mathbf{J}_{f_{K-1}}) \cdot \dots \cdot \det(\mathbf{J}_{f_1})$$

Будем считать определители для каждого шага $\mathbf{f}_k$ и перемножать их:

$$\det\left(\frac{\partial \mathbf{f}}{\partial \mathbf{x}}\right) = \prod_{k=1}^K \det\left(\frac{\partial \mathbf{f}_k}{\partial \mathbf{z}_{k-1}}\right)$$

Обучение нормализующих потоков

Чтобы сделать эту композицию обучаемой, представим f_k как нейросеть с параметрами θ

Плотность, которую моделирует наша модель:

$$p_{\theta}(\mathbf{x}) = p(\mathbf{f}_{\theta}(\mathbf{x})) \left| \det \left(\frac{\partial \mathbf{f}_{\theta}}{\partial \mathbf{x}} \right) \right| = p(\mathbf{z}_K) \left| \prod_{k=1}^K \det \left(\frac{\partial \mathbf{f}_{k,\theta}}{\partial \mathbf{z}_{k-1}} \right) \right|$$

$\mathbf{z}_K$ — финальный вектор в латентном пространстве

Максимизируем правдоподобие:

$$\log p_{\theta}(\mathbf{x}) = \log p(\mathbf{z}_K) + \sum_{k=1}^K \log \left| \det \left(\frac{\partial \mathbf{f}_{k,\theta}}{\partial \mathbf{z}_{k-1}} \right) \right|$$

- $\log p(\mathbf{z}_K)$ отвечает за то, чтобы выход потока соответствовал базовому распределению
- $\sum_{k=1}^K \log \left| \det \left(\frac{\partial \mathbf{f}_{k,\theta}}{\partial \mathbf{z}_{k-1}} \right) \right|$ отслеживает суммарное изменение объема после всех преобразований

План

- Generative Adversarial Networks

- GAN's Problems

- Wasserstein Distance

- Wasserstein GAN

- Развитие GANs

Generative Adversarial Networks

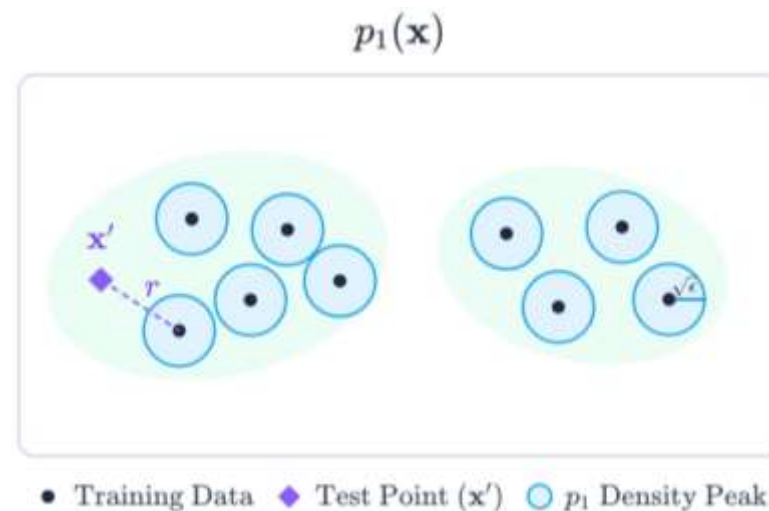
Likelihood based models

- Мы хотим научиться воспроизводить объекты из истинного распределения $p_{data}(\mathbf{x})$
- Обычно мы просто максимизируем правдоподобие $p_{\theta}(\mathbf{x})$

Модель 1:

$$p_1(\mathbf{x}) = \frac{1}{n} \sum_{i=1}^n \mathcal{N}(\mathbf{x}|\mathbf{x}_i, \epsilon \mathbf{I})$$

- Строим узкий гауссовский пик над каждым $\mathbf{x}_i$
- Для малого ϵ образец, почти идентичный одному из обучающих
- На обучающей выборке правдоподобие $\rightarrow \infty$ при $\epsilon \rightarrow 0$
- Для нового объекта $\mathbf{x}'$ правдоподобие $\rightarrow -\infty$



Вывод 1: Хорошие сэмплы не гарантируют высокое правдоподобие

Likelihood based models

Модель 2:

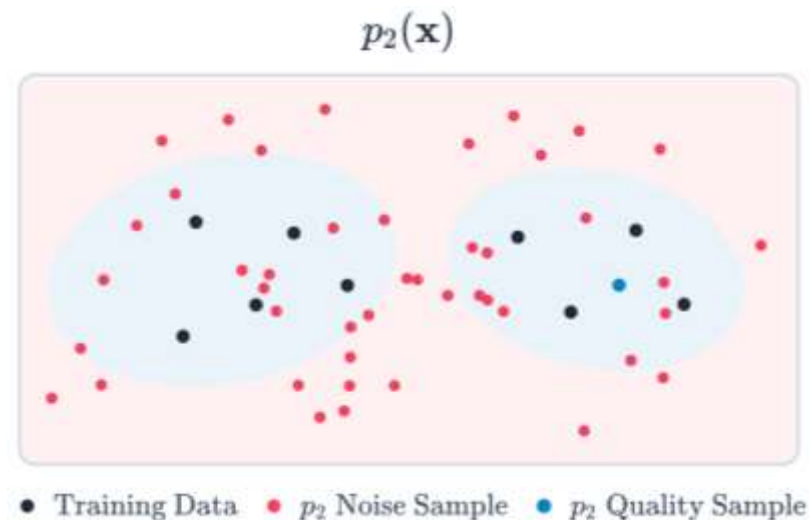
- С вероятностью 0.99 генерирует шум:

$$p_2(\mathbf{x}) = 0.01p(\mathbf{x}) + 0.99p_{noise}(\mathbf{x})$$

- Правдоподобие такой модели:

$$\begin{aligned} \log [0.01p(\mathbf{x}) + 0.99p_{noise}(\mathbf{x})] &\geq \\ \log [0.01p(\mathbf{x})] &= \log p(\mathbf{x}) - \log 100 \end{aligned}$$

- $\log p(\mathbf{x})$ измеряется в тысячах нат, $\log 100 \approx 4.6$ нат



Вывод 2: Высокое правдоподобие не гарантирует хорошие сэмплы

Likelihood based models

- Правдоподобие – не идеальная мера качества для генеративных моделей
 - Правдоподобие часто может быть невычислимым
-
- Это заставляет задуматься о новой идее, свободной от правдоподобия (*likelihood free-learning*) – *идее состязательного обучения*

Likelihood free – learning

- Откажемся от прямой аппроксимации плотности $p_{data}(\mathbf{x})$
- Будем учить генератор G_θ , задающий $p_\theta(\mathbf{x})$, создавать образцы, которые выглядят как будто они взяты из $p_{data}(\mathbf{x})$

У нас будет 2 набора данных:

- **Реальные образцы (real samples)** $\{\mathbf{x}_i\}_{i=1}^{n_1}$ из распределения $p_{data}(\mathbf{x})$
- **Сгенерированные образцы (fake samples)** $\{\mathbf{x}_j\}_{j=1}^{n_2}$ из распределения $p_\theta(\mathbf{x})$

Вводим **классификатор (Discriminator)**, который будет учиться различать эти 2 набора:

- $D(\mathbf{x}) = p(y = 1|\mathbf{x})$ – вероятность того, что образец $\mathbf{x}$ является реальным
- $1 - D(\mathbf{x}) = p(y = 0|\mathbf{x})$ – вероятность того, что образец $\mathbf{x}$ является сгенерированным

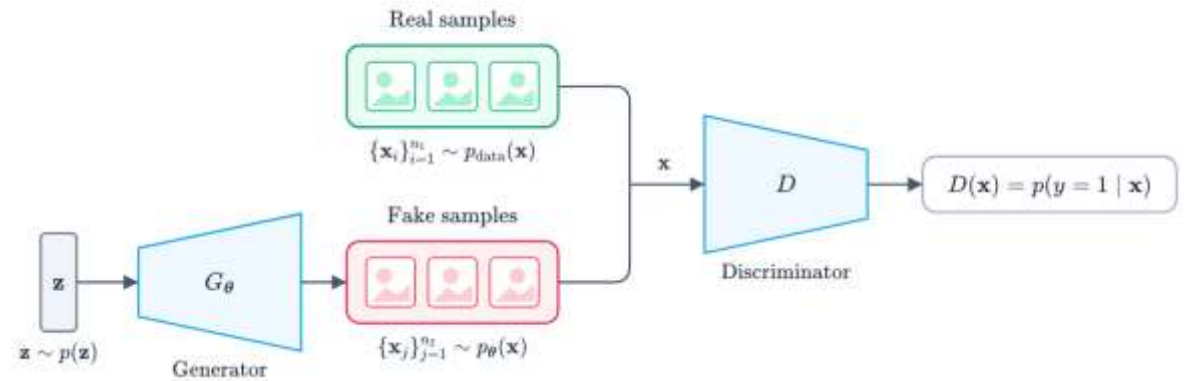
Likelihood free – learning

- Если $p_{\theta}(\mathbf{x}) = p_{data}(\mathbf{x})$, то реальные и сгенерированные объекты встречаются одинаково часто в любой точке $\mathbf{x}$
 - Даже лучший дискриминатор не может отличить такие объекты и всегда выдаёт $D(\mathbf{x}) = \frac{1}{2}$ для любого $\mathbf{x}$
- Генератор идеально изучил истинное распределение $p_{data}(\mathbf{x})$, когда лучший из возможных дискриминаторов не может отличить реальные образцы от сгенерированных
 - Наша цель – достичь этого равновесия
- Модель вообще не вычисляет правдоподобие
 - Качество генератора измеряется тем, насколько трудно отличить его сэмплы от настоящих
 - *GAN* превращает эту идею в процедуру обучения

Generative Adversarial Networks

Обучение GAN реализуется через состязание двух нейросетей:

- **Генератор G_θ :**
создаёт реалистичные данные $\mathbf{x} = G_\theta(\mathbf{z})$
из $\mathbf{z} \sim p(\mathbf{z})$
- **Дискриминатор D :**
бинарный классификатор, который получает на вход объект $\mathbf{x}$ и выдает вероятность $D(\mathbf{x})$, что $\mathbf{x}$ — реальный



Генератор

По аналогии с моделями скрытых переменных $\mathbf{z}$ — это латентный вектор, $p(\mathbf{z})$ — априорное распределение

Мы можем думать о генераторе, как о детерминированном декодере:

$$p_{\theta}(\mathbf{x}, \mathbf{z}) = p_{\theta}(\mathbf{x}|\mathbf{z}) \cdot p(\mathbf{z})$$

где $p_{\theta}(\mathbf{x}|\mathbf{z}) = \delta(\mathbf{x} - G_{\theta}(\mathbf{z}))$

δ — дельта-функция Дирака, для одного $\mathbf{z}$ на выходе вся вероятность сосредоточена в одной точке $\mathbf{x} = G_{\theta}(\mathbf{z})$

Функция потерь

- Мы отказались от правдоподобия $p_{\theta}(\mathbf{x})$, потому что его нельзя вычислить
- D – обычный бинарный классификатор и правдоподобие его ответов $p(y|\mathbf{x})$ легко считается
- Будем обучать D как любой классификатор, который максимизирует вероятность (правдоподобие) присвоения правильных меток всем классам:

$$\begin{aligned} \max_{p(y|\mathbf{x})} (\mathbb{E}_{p_{data}(\mathbf{x})} \log p(y = 1|\mathbf{x}) + \mathbb{E}_{p_{\theta}(\mathbf{x})} \log p(y = 0|\mathbf{x})) = \\ = \max_D (\mathbb{E}_{p_{data}(\mathbf{x})} \log D(\mathbf{x}) + \mathbb{E}_{p_{\theta}(\mathbf{x})} \log(1 - D(\mathbf{x}))) = \max_D V(G, D) \end{aligned}$$

– $V(G, D)$ – обычная **бинарная кросс-энтропия (BCE)**

Функция потерь

Перепишем как минимизацию:

$$\min_D (-\mathbb{E}_{p_{data}(\mathbf{x})} \log D(\mathbf{x}) - \mathbb{E}_{p_{\theta}(\mathbf{x})} \log(1 - D(\mathbf{x})))$$

- $-\mathbb{E}_{p_{data}(\mathbf{x})} \log D(\mathbf{x})$ отвечает за реальные данные, мы хотим $D(\mathbf{x}) \rightarrow 1$
 - Если $D(\mathbf{x}) = 1$, то $-\log 1 = 0$
 - Если $D(\mathbf{x}) = 0.01$, то $-\log 0.01 \approx 4.6$
-
- $-\mathbb{E}_{p_{data}(\mathbf{x})} \log D(\mathbf{x})$ — это штраф за то, что дискриминатор назвал реальный объект сгенерированным

Функция потерь

Перепишем как минимизацию:

$$\min_D (-\mathbb{E}_{p_{data}(\mathbf{x})} \log D(\mathbf{x}) - \mathbb{E}_{p_{\theta}(\mathbf{x})} \log(1 - D(\mathbf{x})))$$

- $-\mathbb{E}_{p_{\theta}(\mathbf{x})} \log(1 - D(\mathbf{x}))$ отвечает за сгенерированные данные, мы хотим $D(\mathbf{x}) \rightarrow 0$
 - Если $D(\mathbf{x}) = 0$, то $-\log 1 = 0$
 - Если $D(\mathbf{x}) = 0.99$, то $-\log 0.01 \approx 4.6$
-
- $-\mathbb{E}_{p_{\theta}(\mathbf{x})} \log(1 - D(\mathbf{x}))$ – это штраф за то, что дискриминатор назвал сгенерированный объект реальным

Минимаксная игра

Генератор хочет, чтобы D ошибался, и минимизирует ту же функцию:

$$\min_G \max_D (\mathbb{E}_{p_{data}(\mathbf{x})} \log D(\mathbf{x}) + \mathbb{E}_{p_{\theta}(\mathbf{x})} \log(1 - D(\mathbf{x})))$$

Сэмплировать из $p_{\theta}(\mathbf{x})$ — значит взять $\mathbf{z} \sim p(\mathbf{z})$ и вычислить $G(\mathbf{z})$:

$$\min_G \max_D (\mathbb{E}_{p_{data}(\mathbf{x})} \log D(\mathbf{x}) + \mathbb{E}_{p(\mathbf{z})} \log(1 - D(G(\mathbf{z}))))$$

- $\max_D$ выведен строго из ММП для классификатора
- $\min_G$ — выбор авторов GAN

Оптимальный дискриминатор

Найдем оптимальный дискриминатор D^* при любом фиксированном генераторе G

Хотим найти D , который максимизирует $V(G, D)$:

$$V(G, D) = \mathbb{E}_{p_{data}(\mathbf{x})} \log D(\mathbf{x}) + \mathbb{E}_{p_{\theta}(\mathbf{x})} \log(1 - D(\mathbf{x})) = \int [p_{data}(\mathbf{x}) \log D(\mathbf{x}) + p_{\theta}(\mathbf{x}) \log(1 - D(\mathbf{x}))] d\mathbf{x}$$

Найдем максимум подынтегрального выражения:

$$\frac{\partial}{\partial D} [p_{data}(\mathbf{x}) \log D(\mathbf{x}) + p_{\theta}(\mathbf{x}) \log(1 - D(\mathbf{x}))] = \frac{p_{data}(\mathbf{x})}{D(\mathbf{x})} - \frac{p_{\theta}(\mathbf{x})}{1 - D(\mathbf{x})} = 0 \quad \Rightarrow \quad D^*(\mathbf{x}) = \frac{p_{data}(\mathbf{x})}{p_{data}(\mathbf{x}) + p_{\theta}(\mathbf{x})}$$

Цель генератора

Найдем цель генератора при условии оптимального дискриминатора D^*

Введём обозначения:

$$1 - D^*(\mathbf{x}) = \frac{p_\theta(\mathbf{x})}{p_\theta(\mathbf{x}) + p_{data}(\mathbf{x})}, \quad m(\mathbf{x}) = \frac{p_{data}(\mathbf{x}) + p_\theta(\mathbf{x})}{2}, \quad JSD = \frac{1}{2}KL(p||m) + \frac{1}{2}KL(m||q)$$

Подставим D^* в нашу функцию $V(G, D)$ и посмотрим, что минимизирует генератор:

$$\begin{aligned} V(G, D^*) &= \int p_{data}(\mathbf{x}) \log D^*(\mathbf{x}) d\mathbf{x} + \int p_\theta(\mathbf{x}) \log(1 - D^*(\mathbf{x})) d\mathbf{x} = \\ &= \int p_{data}(\mathbf{x}) \log \frac{p_{data}(\mathbf{x})}{2m(\mathbf{x})} d\mathbf{x} + \int p_\theta(\mathbf{x}) \log \left(\frac{p_\theta(\mathbf{x})}{2m(\mathbf{x})} \right) d\mathbf{x} = \int p_{data}(\mathbf{x}) \log \frac{p_{data}(\mathbf{x})}{m(\mathbf{x})} d\mathbf{x} - \log 2 \int p_{data}(\mathbf{x}) d\mathbf{x} \\ &+ \int p_\theta(\mathbf{x}) \log \frac{p_\theta(\mathbf{x})}{m(\mathbf{x})} d\mathbf{x} - \log 2 \int p_\theta(\mathbf{x}) d\mathbf{x} = KL(p_{data}(\mathbf{x}) || m(\mathbf{x})) + KL(p_\theta(\mathbf{x}) || m(\mathbf{x})) - 2 \log 2 \\ &= 2JSD(p_{data}(\mathbf{x}) || p_\theta(\mathbf{x})) - 2 \log 2 \end{aligned}$$

Оптимальный GAN

Дивергенция Йенсена-Шеннона:

$$JSD(p_{data}(\mathbf{x}) || p_{\theta}(\mathbf{x})) = \frac{1}{2} \left[KL \left(p_{data}(\mathbf{x}) || \frac{p_{data}(\mathbf{x}) + p_{\theta}(\mathbf{x})}{2} \right) + KL \left(p_{\theta}(\mathbf{x}) || \frac{p_{data}(\mathbf{x}) + p_{\theta}(\mathbf{x})}{2} \right) \right]$$

Истинная цель генератора – минимизация дивергенции Йенсена-Шеннона:

$$2JSD(p_{data}(\mathbf{x}) || p_{\theta}(\mathbf{x})) - 2 \log 2 \rightarrow \min_{\theta}$$

- $JSD \geq 0$
- $JSD(p_{data}(\mathbf{x}) || p_{\theta}(\mathbf{x})) = 0$ когда $p_{data}(\mathbf{x}) = p_{\theta}(\mathbf{x})$

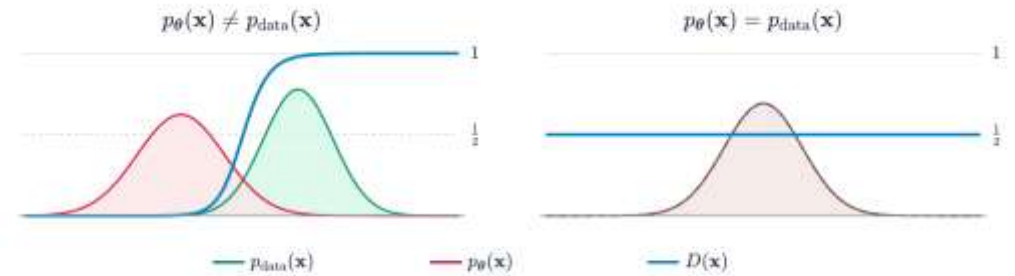
Минимум по генератору достигается при $p_{data}(\mathbf{x}) = p_{\theta}(\mathbf{x})$ – генератор идеально воспроизводит данные

Точка равновесия

Подставим $p_{data}(\mathbf{x}) = p_{\theta}(\mathbf{x})$ в $D^*(\mathbf{x})$ и V :

$$D^*(\mathbf{x}) = \frac{p_{data}(\mathbf{x})}{p_{data}(\mathbf{x}) + p_{\theta}(\mathbf{x})} = \frac{1}{2}$$

$$V(G^*, D^*) = 2JSD(p_{data}(\mathbf{x}) || p_{\theta}(\mathbf{x})) - 2 \log 2 = -2 \log 2$$



Равновесие:

- Генератор идеально воспроизводит данные, а дискриминатор не может отличить реальные объекты от сгенерированных

Но вывод сделан на предположении, что D — **любая функция** и каждый раз доучивается до оптимума

Обучение GAN

На практике G и D — нейросети с параметрами θ и ϕ :

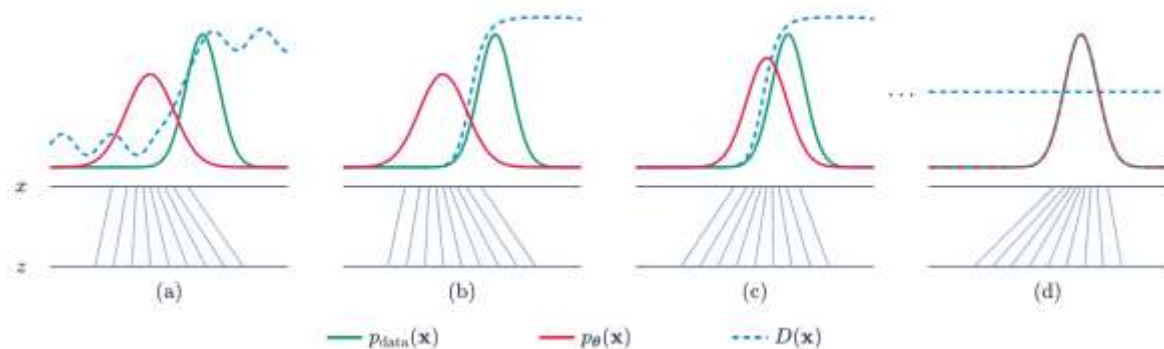
$$\min_G \max_D V(G, D) = \min_{\theta} \max_{\phi} [\mathbb{E}_{p_{data}(\mathbf{x})} \log D_{\phi}(\mathbf{x}) + \mathbb{E}_{p(\mathbf{z})} \log(1 - D_{\phi}(G_{\theta}(\mathbf{z})))]$$

Discriminator:

- Фиксируем веса генератора θ
- Делаем шаг градиентного подъема по ϕ , чтобы максимизировать $V(G, D)$

Generator:

- Фиксируем веса дискриминатора ϕ
- Делаем шаг градиентного спуска по θ , чтобы минимизировать $V(G, D)$



Проблемы GAN

Теоретическое равновесие требует, чтобы модели G и D были любыми функциями, а дискриминатор каждый раз доучивался до оптимума

В действительности G и D — нейросети с конечным набором параметров и между шагами G мы делаем лишь несколько шагов обучения D

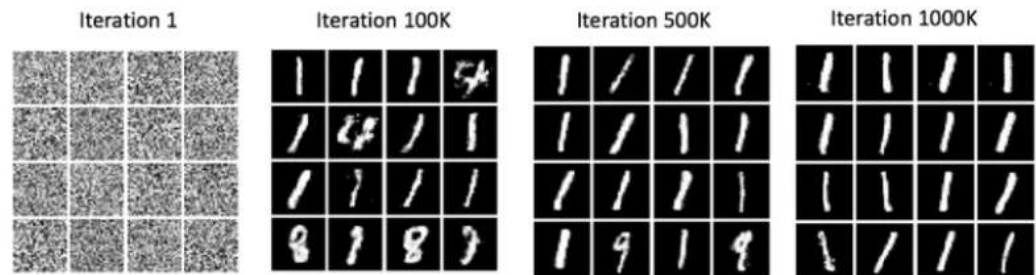
- Из-за этой ограниченной мощности настоящее теоретическое равновесие для многих архитектур GAN может не существовать
- Это и является основной причиной многих проблем при обучении GAN

GAN's Problems

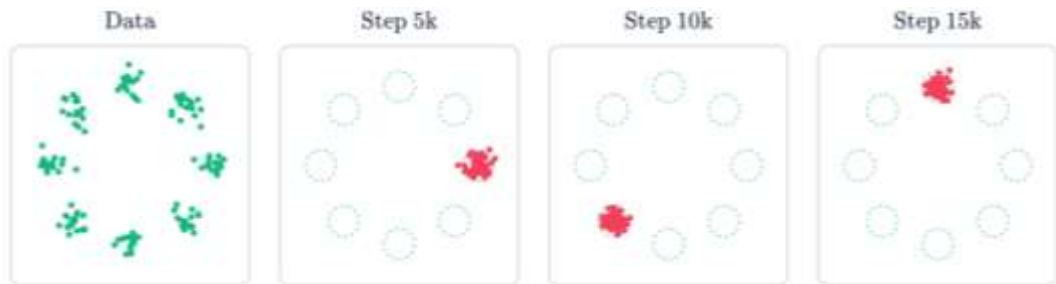
Mode Collapse

Коллапс мод (mode collapse) – главная практическая проблема GAN

Генератор «схлопывается» и начинает генерировать только одну или несколько мод распределения



- D оценивает объекты отдельно, однообразие не наказывается
- Генератору выгодно рисовать то, что D сейчас считает реальным и перескакивать с моды на моду



Более глубокая причина – дивергенция, которую минимизирует GAN

Jensen – Shannon vs Kullback – Leibler

Пусть:

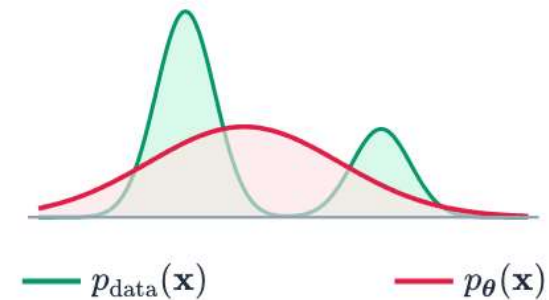
- $p_{data}(\mathbf{x})$ – истинное распределение, смесь двух гауссиан
- $p_{\theta}(\mathbf{x}) = \mathcal{N}(\mathbf{x} | \mu, \sigma^2)$ – модель с одной модой

Forward KL (mode covering):

$$KL(p_{data} || p_{\theta}) = \int p_{data}(\mathbf{x}) \log \frac{p_{data}(\mathbf{x})}{p_{\theta}(\mathbf{x})} d\mathbf{x}$$

- Интеграл взвешен $p_{data}(\mathbf{x})$, поэтому важны области, где есть данные
- Если в этих областях $p_{\theta} \rightarrow 0$, штраф $\rightarrow \infty$
- Чтобы избежать ∞ , модель вынуждена покрывать все моды
- Будем попадать между модами, где нет данных $\rightarrow$ размытые сэмплы

$KL(p_{data} || p_{\theta})$
Forward KL, mode covering

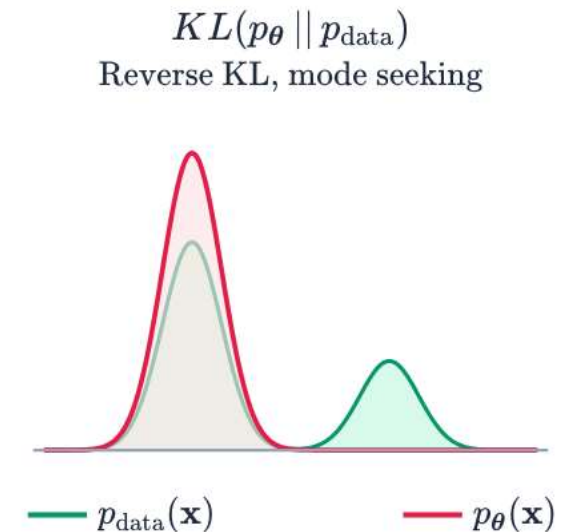


Jensen – Shannon vs Kullback – Leibler

Reverse KL (mode seeking):

$$KL(p_{\theta} || p_{data}) = \int p_{\theta}(\mathbf{x}) \log \frac{p_{\theta}(\mathbf{x})}{p_{data}(\mathbf{x})} d\mathbf{x}$$

- Интеграл взвешен $p_{\theta}(\mathbf{x})$, поэтому важны области, куда модель кладёт вероятность
- Если в этих областях $p_{data} \rightarrow 0$, штраф $\rightarrow \infty$
- Чтобы избежать ∞ , модель вынуждена избегать места, где $p_{data}(\mathbf{x}) \rightarrow 0$
- Если сесть на одну моду, то на пропущенной моде $p_{\theta}(\mathbf{x}) \approx 0$ и штраф маленький
- Если растянуться, то $p_{data} \rightarrow 0$ и $p_{\theta}(\mathbf{x}) > 0$, штраф $\rightarrow \infty$



Jensen – Shannon vs Kullback – Leibler

JSD (mode seeking):

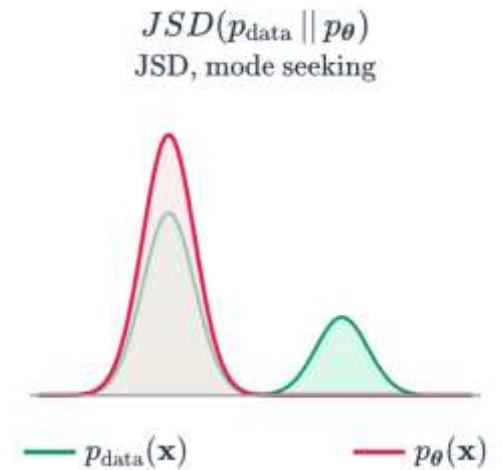
$$JSD(p_{data} || p_{\theta}) = \frac{1}{2} \int p_{data}(\mathbf{x}) \log \frac{p_{data}(\mathbf{x})}{m(\mathbf{x})} d\mathbf{x} + \frac{1}{2} \int p_{\theta}(\mathbf{x}) \log \frac{p_{\theta}(\mathbf{x})}{m(\mathbf{x})} d\mathbf{x}$$

Оценим дробь под логарифмом:

$$m(\mathbf{x}) = \frac{p_{data}(\mathbf{x}) + p_{\theta}(\mathbf{x})}{2} \geq \frac{p_{data}(\mathbf{x})}{2} \Rightarrow \frac{p_{data}(\mathbf{x})}{m(\mathbf{x})} \leq 2, \quad \frac{p_{\theta}(\mathbf{x})}{m(\mathbf{x})} \leq 2$$

Подставим:

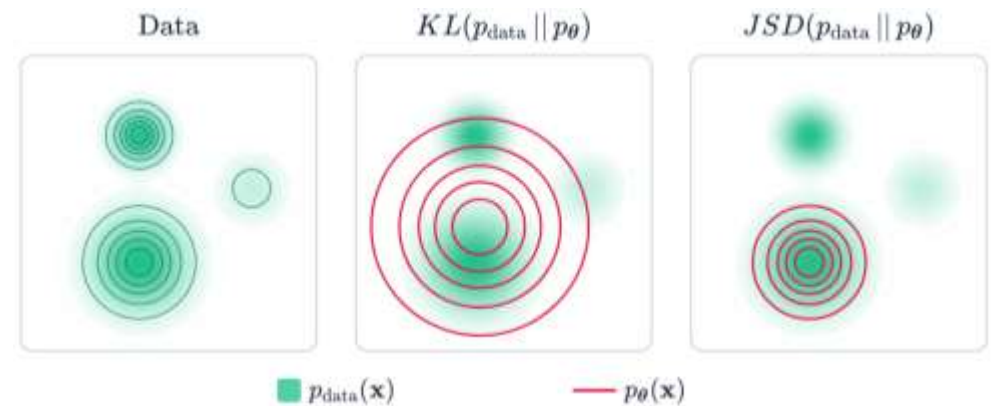
$$JSD \leq \frac{1}{2} \int p_{data}(\mathbf{x}) \log 2 d\mathbf{x} + \frac{1}{2} \int p_{\theta}(\mathbf{x}) \log 2 d\mathbf{x} = \log 2$$



Штраф за пропущенную моду конечен и сесть на одну моду оказывается выгоднее, чем растянуться на все

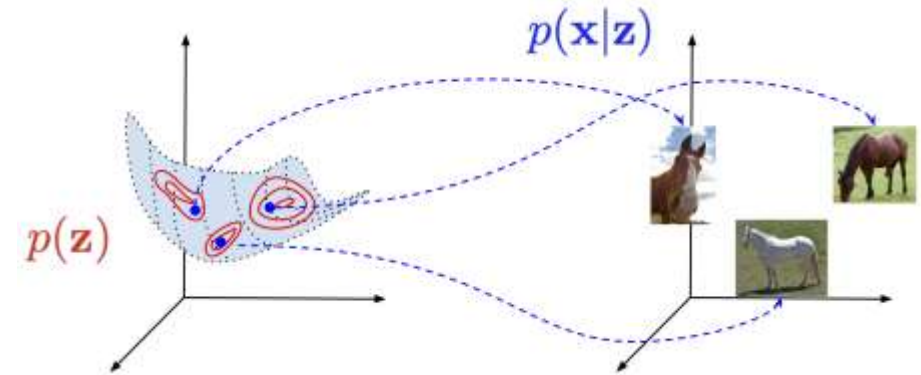
Итог JSD vs KLD

- Генератору проще найти одну моду и идеально её воспроизводить
- GAN дают чёткие, но менее разнообразные объекты
- Модели правдоподобия разнообразные, но размытые



Гипотеза о многообразии

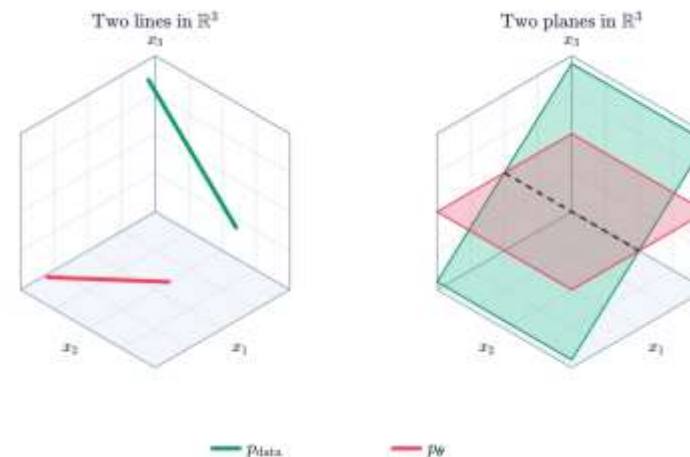
- В пространствах высокой размерности почти каждая точка – это шум
- Многообразие – это небольшая «осмысленная» область внутри этого пространства



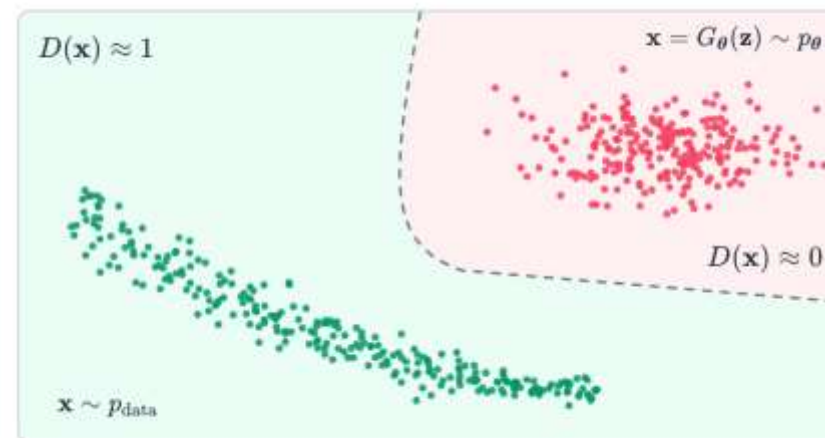
- Многие многомерные данные, которые встречаются в реальном мире, лежат вдоль низкоразмерных скрытых многообразий внутри этого многомерного пространства (*manifold hypothesis*)
- Сгенерированные объекты также имеют низкую размерность, поскольку создаются из низкоразмерного источника $\mathbf{z}$

Непересекающиеся носители

- Носитель – множество точек, где плотность > 0
- В высокой размерности p_{data} и p_{θ} почти гарантированно не пересекаются (*disjoint supports*)



- Дискриминатору очень легко провести разделяющую поверхность и стать идеальным классификатором
- $D(\mathbf{x}) \approx 1$ на реальных, $D(\mathbf{x}) \approx 0$ на сгенерированных данных



KL при непересекающихся носителях

Forward KL:

- В случае непересекающихся носителей в области, где $p_{data}(\mathbf{x}) > 0$, мы имеем $p_{\theta}(\mathbf{x}) = 0$:

$$KL(p_{data}||p_{\theta}) = \int p_{data}(\mathbf{x}) \log \frac{p_{data}(\mathbf{x})}{p_{\theta}(\mathbf{x})} d\mathbf{x} \rightarrow \infty$$

Reverse KL:

- В случае непересекающихся носителей в области, где $p_{\theta}(\mathbf{x}) > 0$, мы имеем $p_{data}(\mathbf{x}) = 0$:

$$KL(p_{\theta}||p_{data}) = \int p_{\theta}(\mathbf{x}) \log \frac{p_{\theta}(\mathbf{x})}{p_{data}(\mathbf{x})} d\mathbf{x} \rightarrow \infty$$

- Обе KL бесконечны при любом положении генератора $\rightarrow$ полезного сигнала для обучения нет

JSD при непересекающихся носителях

Jensen – Shannon:

$$\begin{aligned} JSD(p_{data} \parallel p_{\theta}) &= \frac{1}{2} KL \left(p_{data}(\mathbf{x}) \parallel \frac{p_{data}(\mathbf{x}) + p_{\theta}(\mathbf{x})}{2} \right) + \frac{1}{2} KL \left(p_{\theta}(\mathbf{x}) \parallel \frac{p_{data}(\mathbf{x}) + p_{\theta}(\mathbf{x})}{2} \right) = \\ &= \frac{1}{2} \int p_{data}(\mathbf{x}) \log \frac{p_{data}(\mathbf{x})}{m(\mathbf{x})} d\mathbf{x} + \frac{1}{2} \int p_{\theta}(\mathbf{x}) \log \frac{p_{\theta}(\mathbf{x})}{m(\mathbf{x})} d\mathbf{x} = \frac{1}{2} \int p_{data}(\mathbf{x}) \log \frac{p_{data}(\mathbf{x})}{p_{data}(\mathbf{x})/2} d\mathbf{x} + \\ &+ \frac{1}{2} \int p_{\theta}(\mathbf{x}) \log \frac{p_{\theta}(\mathbf{x})}{p_{\theta}(\mathbf{x})/2} d\mathbf{x} = \frac{1}{2} \log 2 + \frac{1}{2} \log 2 = \log 2 \end{aligned}$$

- Как только дискриминатор становится идеальным, JSD становится константой
- Обучение останавливается, поскольку градиенты становятся равными нулю

Затухание градиентов

- Идеальный D выдаёт ≈ 0 на всех сгенерированных объектах:

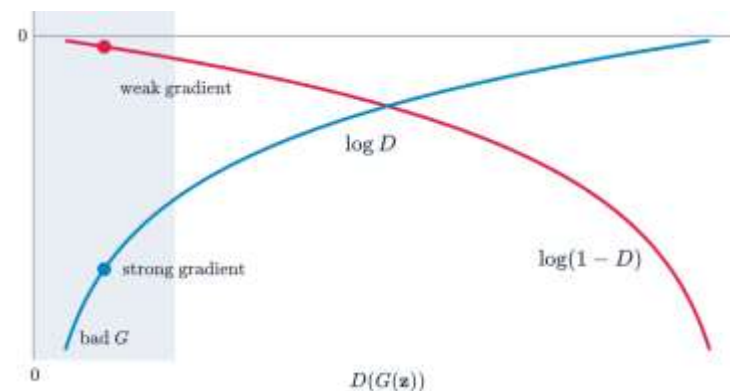
$$\min_G \max_D \left(\mathbb{E}_{p_{data}(\mathbf{x})} \log D(\mathbf{x}) + \mathbb{E}_{p(\mathbf{z})} \log \left(1 - D(G(\mathbf{z})) \right) \right)$$

- Градиент от $\log \left(1 - D(G(\mathbf{z})) \right) \approx 0$ и генератор не получает подсказки куда двигаться (*vanishing gradients*)
- **Парадокс:** чем лучше обучен дискриминатор, тем хуже учится генератор

- Частичное решение – *non-saturating loss*:

$$\min_G \mathbb{E}_{p(\mathbf{z})} \log \left(1 - D(G(\mathbf{z})) \right) \rightarrow \max_G \mathbb{E}_{p(\mathbf{z})} \log \left(D(G(\mathbf{z})) \right)$$

- Цель та же, но при $D(G(\mathbf{z})) \approx 0$ градиент больше



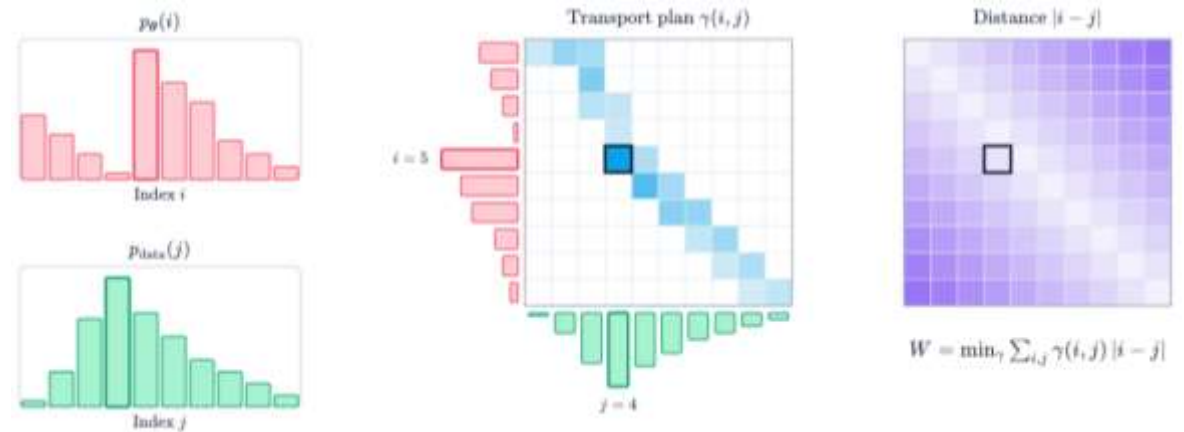
Wasserstein Distance

Wasserstein Distance

- Мы можем представить два распределения как две кучи земли
- Стоимость перемещения рассчитывается следующим образом:

$$\text{cost} = \text{amount} \times \text{distance}$$

- Одну кучу можно переложить разными способами (**транспортными планами**)



Расстоянием Вассерштейна (Wasserstein Distance, Earth Mover's Distance) будем называть стоимость самого эффективного транспортного плана γ :

$$W = \min_{\gamma} \sum_{(i,j)} \gamma(i, j) |i - j|$$

Wasserstein Distance

В непрерывном случае расстояние Вассерштейна определяется следующим образом:

$$W(p_{data}, p_{\theta}) = \inf_{\gamma \in \Gamma(p_{data}, p_{\theta})} \mathbb{E}_{(\mathbf{x}, \mathbf{y}) \sim \gamma} \|\mathbf{x} - \mathbf{y}\| = \inf_{\gamma \in \Gamma(p_{data}, p_{\theta})} \int \|\mathbf{x} - \mathbf{y}\| \gamma(\mathbf{x}, \mathbf{y}) d\mathbf{x} d\mathbf{y}$$

- γ – совместное распределение, показывающее сколько массы нужно перенести из $\mathbf{x}$ в $\mathbf{y}$, чтобы одно распределение превратилось в другое (**транспортный план**)
- $\Gamma(p_{data}(\mathbf{x}), p_{\theta})$ – множество всех возможных транспортных планов γ
- $\mathbb{E}_{(\mathbf{x}, \mathbf{y}) \sim \gamma} \|\mathbf{x} - \mathbf{y}\|$ – ожидаемая стоимость перемещения массы из точки $\mathbf{x}$ в точку $\mathbf{y}$ согласно плану γ

Существует целое семейство **метрик Вассерштейна**:

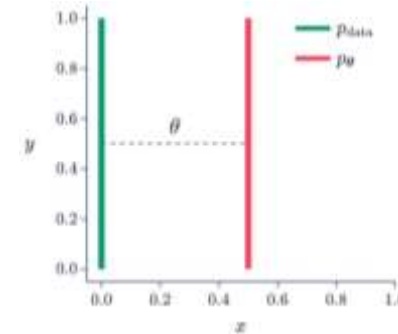
$$W_s(p_{data}, p_{\theta}) = \inf_{\gamma \in \Gamma(p_{data}, p_{\theta})} \left(\mathbb{E}_{(\mathbf{x}, \mathbf{y}) \sim \gamma} \|\mathbf{x} - \mathbf{y}\|^s \right)^{\frac{1}{s}}$$

Мы будем использовать метрику $W_1(p_{data}, p_{\theta})$, то есть когда $s = 1$

Wasserstein Distance vs KL vs JSD

Рассмотрим 2 распределения p_{data} и p_{θ} :

$$p_{data}(x, y) = (0, U[0,1])$$
$$p_{\theta}(x, y) = (\theta, U[0,1])$$



При $\theta = 0$ распределения совпадают:

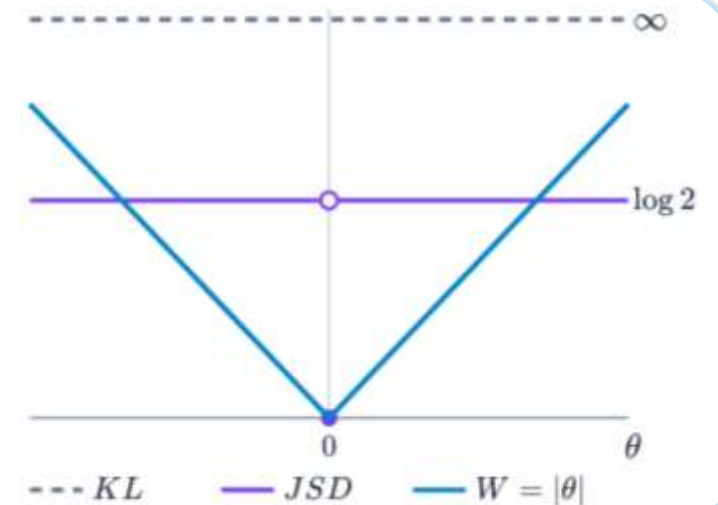
$$KL(p_{data}||p_{\theta}) = KL(p_{\theta}||p_{data}) = JSD(p_{data}||p_{\theta}) = W(p_{data}, p_{\theta}) = 0$$

При $\theta \neq 0$:

$$KL(p_{data}||p_{\theta}) = \int_{U[0,1]} 1 \log \frac{1}{0} dy = \infty$$

$$JSD(p_{data}||p_{\theta}) = \frac{1}{2} \left(\int_{U[0,1]} 1 \log \frac{1}{1/2} dy + \int_{U[0,1]} 1 \log \frac{1}{1/2} dy \right) = \log 2$$

$$W(p_{data}, p_{\theta}) = |\theta|$$



Wasserstein Distance vs KL vs JSD

Вывод:

- Расстояние Вассерштейна – непрерывная и почти всюду дифференцируемая функция, даже если носители не пересекаются (**теорема 1**)
- Оно всегда даёт осмысленный градиент, который генератор использует во время обучения

Теорема 2:

- Сходимость по W более слабое условие, чем сходимость по JSD или KL
- Даже если обучение не сходится по JSD , оно может сойтись по W , что делает обучение более стабильным

Wasserstein GAN

Теорема Канторовича-Рубинштейна

Проблема:

- Вычислить W напрямую через $\inf$ невозможно:

$$W(p_{data}, p_{\theta}) = \inf_{\gamma \in \Gamma(p_{data}, p_{\theta})} \mathbb{E}_{(\mathbf{x}, \mathbf{y}) \sim \gamma} \|\mathbf{x} - \mathbf{y}\|$$

Будем использовать **теорему Канторовича–Рубинштейна** и вместо лучшего плана γ искать лучшую функцию f :

$$W(p_{data}, p_{\theta}) = \frac{1}{K} \max_{\|f\|_L \leq K} [\mathbb{E}_{p_{data}} f(\mathbf{x}) - \mathbb{E}_{p_{\theta}} f(\mathbf{x})]$$

- Это работает только для функций f , которые являются K –Липшицевыми:

$$|f(\mathbf{x}_1) - f(\mathbf{x}_2)| \leq K \|\mathbf{x}_1 - \mathbf{x}_2\|$$

- Это означает, что крутизна (градиент) функции f ограничена константой K

Wasserstein GAN

Мы не можем найти идеальную f , поэтому будем аппроксимировать ее с помощью нейросети критика f_ϕ (**Critic**)

Обеспечение Липшицевости:

- Будем принудительно обрезать (**clipping**) все веса в диапазон $[-c, c]$ после каждого шага градиентного спуска
- Это гарантирует, что функция будет K – Липшицевой

Множество наших нейросетей f_ϕ – это подмножество всех K – Липшицевых функций f :

$$K \cdot W(p_{data}, p_\theta) = \max_{\|f\|_L \leq K} [\mathbb{E}_{p_{data}} f(\mathbf{x}) - \mathbb{E}_{p_\theta} f(\mathbf{x})] \geq \max_{\phi \in \Phi} [\mathbb{E}_{p_{data}} f_\phi(\mathbf{x}) - \mathbb{E}_{p_\theta} f_\phi(\mathbf{x})]$$

Wasserstein GAN

Функция потерь **GAN**:

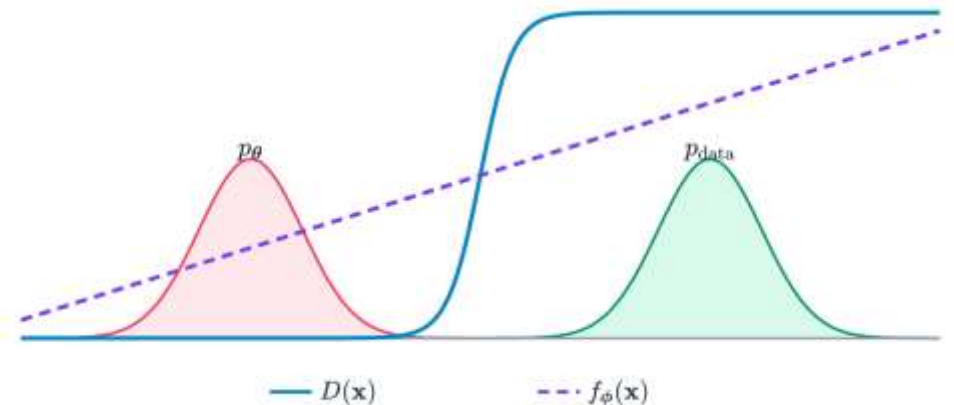
$$\min_{\theta} \max_{\phi} \left(\mathbb{E}_{p_{data}(\mathbf{x})} \log D_{\phi}(\mathbf{x}) + \mathbb{E}_{p(\mathbf{z})} \log \left(1 - D_{\phi}(G_{\theta}(\mathbf{z})) \right) \right)$$

Функция потерь **WGAN**:

$$\min_{\theta} W(p_{data}, p_{\theta}) \approx \min_{\theta} \max_{\phi \in \Phi} \left[\mathbb{E}_{p_{data}(\mathbf{x})} f_{\phi}(\mathbf{x}) - \mathbb{E}_{p(\mathbf{z})} f_{\phi}(G_{\theta}(\mathbf{z})) \right]$$

Основные отличия:

- Дискриминатор $D_{\phi} \rightarrow$ Критик f_{ϕ} выдает любое число (оценку)
- Параметры критика ϕ ограничены



WGAN – GP

Weight Clipping – очень плохой способ заставить критика быть K –Липшицевым и результат зависит от выбранного c

Идея **WGAN – GP**:

Вместо того, чтобы обрезать веса ϕ , будем наказывать критика, если он нарушает 1 –Липшицево ограничение

1 –Липшицевость означает, что норма градиента критика $||\nabla f_{\phi}(\mathbf{x})|| \leq 1$ *в любой точке*

- Мы не можем проверять градиент везде
- Авторы показали, что достаточно проверить его на случайных точках между реальными x_{data} и поддельными x_{fake} данными

WGAN – GP

Gradient Penalty:

- Создаем случайный объект с помощью интерполяции:

$$\hat{\mathbf{x}} = \epsilon \cdot \mathbf{x}_{data} + (1 - \epsilon) \cdot \mathbf{x}_{fake}, \quad \epsilon \sim U[0,1]$$

- Вычисляем градиент критика f_{ϕ} в этой точке $\hat{\mathbf{x}}$ и штрафует его если $\|\nabla f_{\phi}(\hat{\mathbf{x}})\|_2$ отличается от 1

Функция потерь *WGAN – GP*:

$$\mathcal{L} = \underbrace{\left[\mathbb{E}_{p(\mathbf{z})} f_{\phi}(G_{\theta}(\mathbf{z})) - \mathbb{E}_{p_{data}(\mathbf{x})} f_{\phi}(\mathbf{x}) \right]}_{WGAN\ Loss} + \underbrace{\lambda \cdot \mathbb{E}_{\hat{\mathbf{x}}} \left[\left(\|\nabla f_{\phi}(\hat{\mathbf{x}})\|_2 - 1 \right)^2 \right]}_{Gradient\ Penalty}$$

Развитие *GANs*

GAN's Problems

WGAN дал нам стабильные градиенты, но создание *HR*-изображений (1024×1024) все равно сложная задача

Проблема 1:

- Чем выше разрешение, тем больше пространство, в котором живут объекты
- В таком пространстве наши два многообразия почти гарантированно не пересекаются

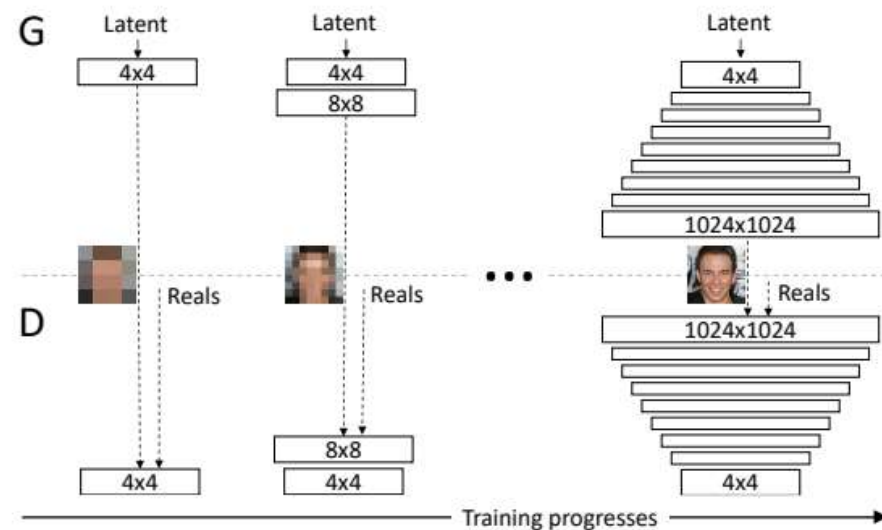
Проблема 2:

- Генерация *HR*-изображений требует огромных моделей, которые занимают всю память
- Мы вынуждены использовать небольшие батчи, обучение на которых даёт шумные и нестабильные градиенты

Progressive Growing GAN

Идея: Будем постепенно увеличивать разрешение генерируемого изображения

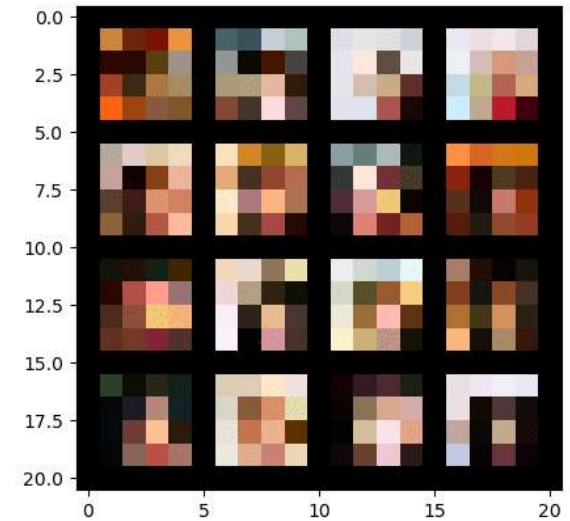
- Обучаем модель генерировать мелкие изображения 4×4
- Добавляем новые слои к обеим сетям (*Upsampling*)
- Плавно подключаем новые слои
- Повторяем процесс $8 \times 8 \rightarrow 16 \times 16 \dots \rightarrow 1024 \times 1024$



Это хорошо стабилизирует обучение:

Модель сначала изучает общую структуру (поза, форма лица) на низких, а затем добавляет мелкие детали на высоких разрешениях

Progressive Growing GAN



GAN's Problems

- ***PG – GAN*** позволил генерировать изображения в высоком разрешении, но он все еще использовали свёрточные сети
- Свёрточные сети по своей природе локальны

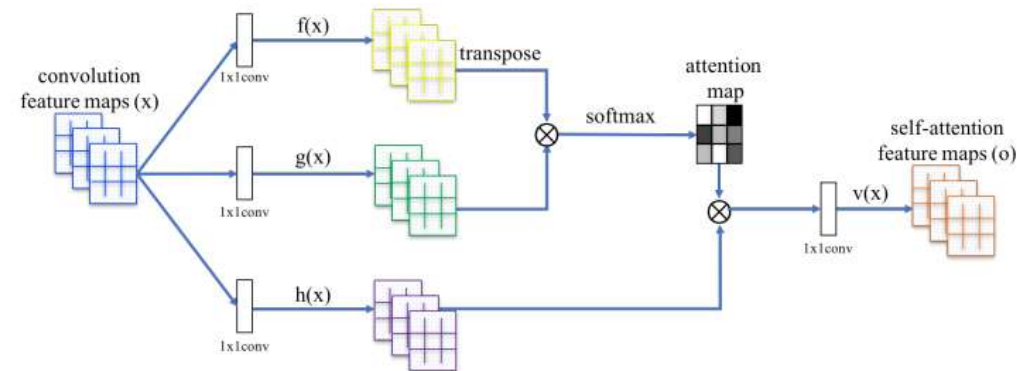
Проблемы:

- Требуются глубокие сети, чтобы моделировать дальние зависимости
- Через много слоев информация о каких-то признаках забывается

Self – Attention GAN

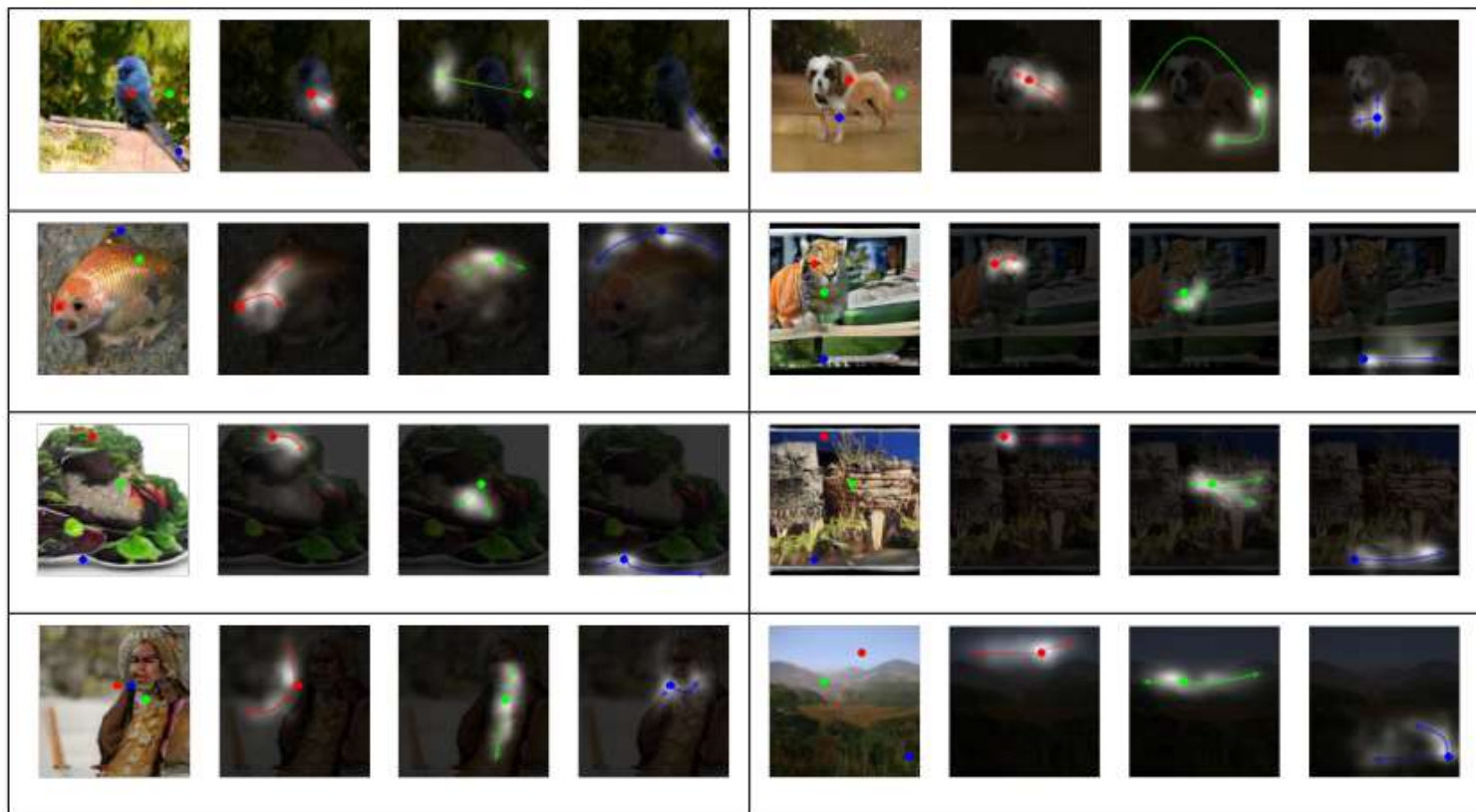
Идея:

- Дадим возможность пикселям смотреть друг на друга (**self – attention**)



- Используется **Hinge Loss**, который тоже решает проблемы затухания градиентов *JSD*
- Липшицевость критика достигается с помощью спектральной нормализации (**Spectral Normalization**)
- Дискриминатор и генератор обучаются с разной скоростью (**Separate Learning Rate**)

Self – Attention GAN



StyleGAN

Идея:

- Возьмем за основу $PG - GAN$ и научим модель делать контролируемую генерацию

Проблема:

- В обычных $GANs$ латентный вектор $\mathbf{z}$ — это черный ящик
- Хотим сделать распутанное пространство, где мы смогли бы контролировать конкретные черты изображения

В *StyleGAN* предложили разделить черты на 3 уровня:

- **Грубые (Coarse)** ($4 \times 4, 8 \times 8$) — поза, форма лица
- **Средние (Middle)** ($16 \times 16, 32 \times 32$) — черты лица, детали прически
- **Мелкие (Fine)** ($64 \times 64 \rightarrow 1024 \times 1024$) — микро-детали, цвет глаз, морщины

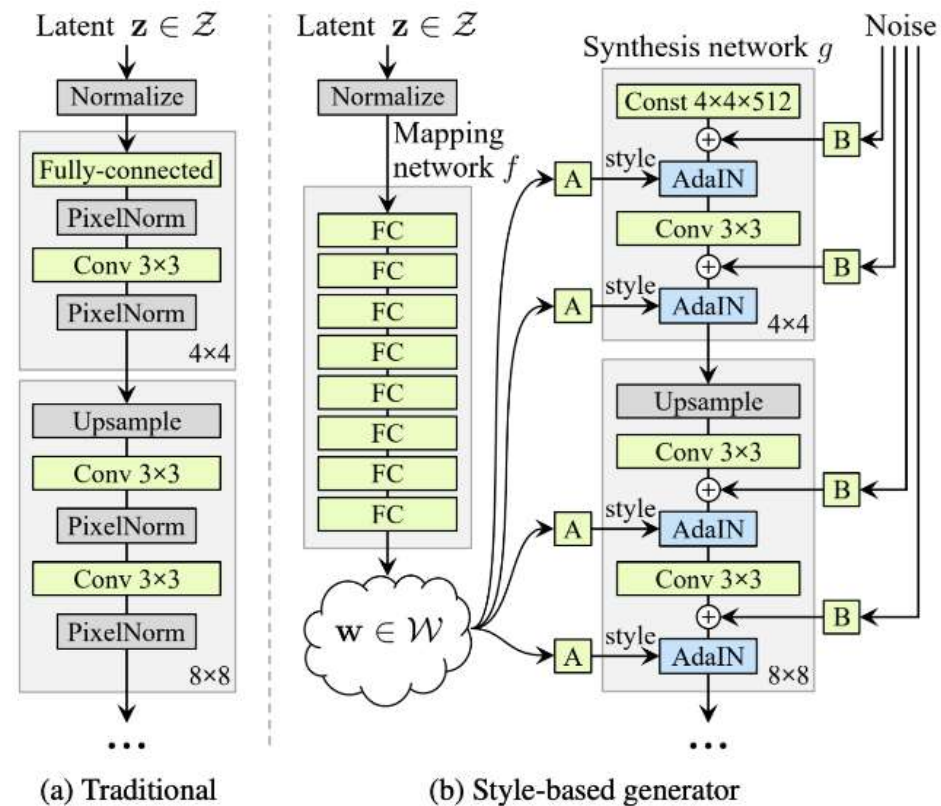
StyleGAN

Обычный GAN:

- Латентный вектор z подается на вход первому слою и проходит через всю сеть перемешиваясь

Style – based GAN:

- Генератор разделён на 2 сети
- **Mapping Network** – 8 слойный *MLP*, который распутывает z и превращает его в стилевой вектор w
- **Synthesis Network** – сверточная сеть, которая рисует изображение, используя w как инструкцию

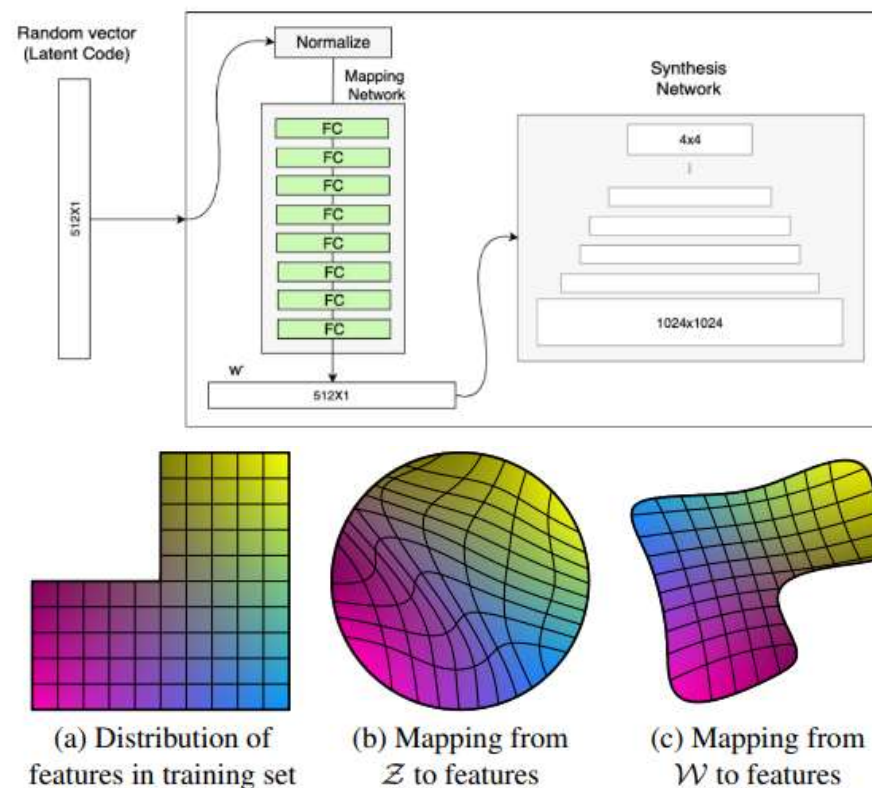


StyleGAN: Mapping Network

Цель:

- Хотим, чтобы каждая компонента вектора z отвечала за некоторую черту

- В реальности z следует обучающим данным
- Будем использовать **Mapping Network**, которая будет распутывать z и превращать его в стиливой вектор w
- **Synthesis Network** будет требовать от **Mapping Network** понятных инструкций

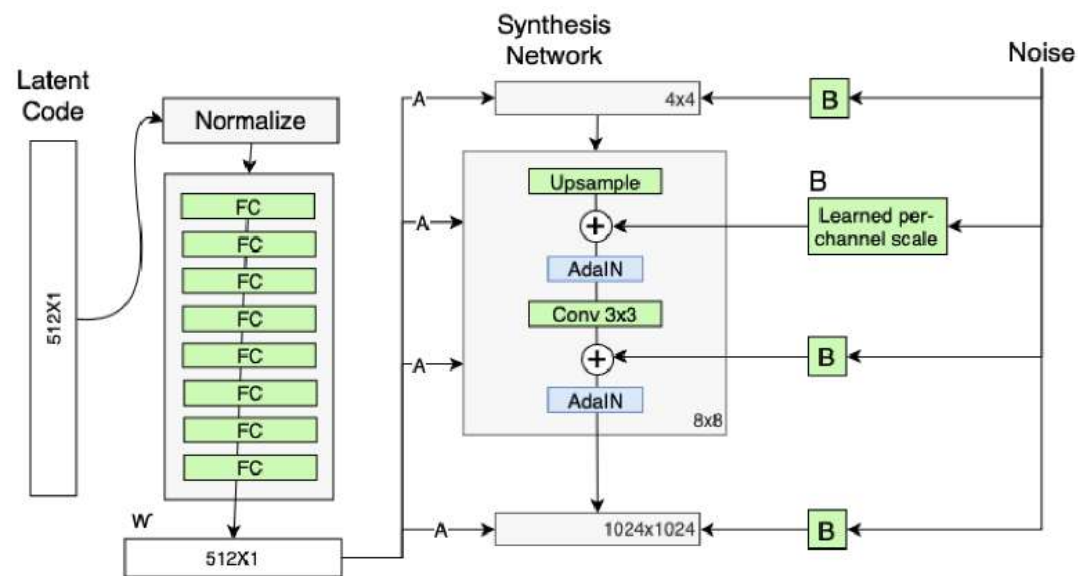


StyleGAN: Stochastic Variation

Вектор w контролирует все главные черты, но мы хотим сделать лицо более реалистичным со случайными деталями

Решение:

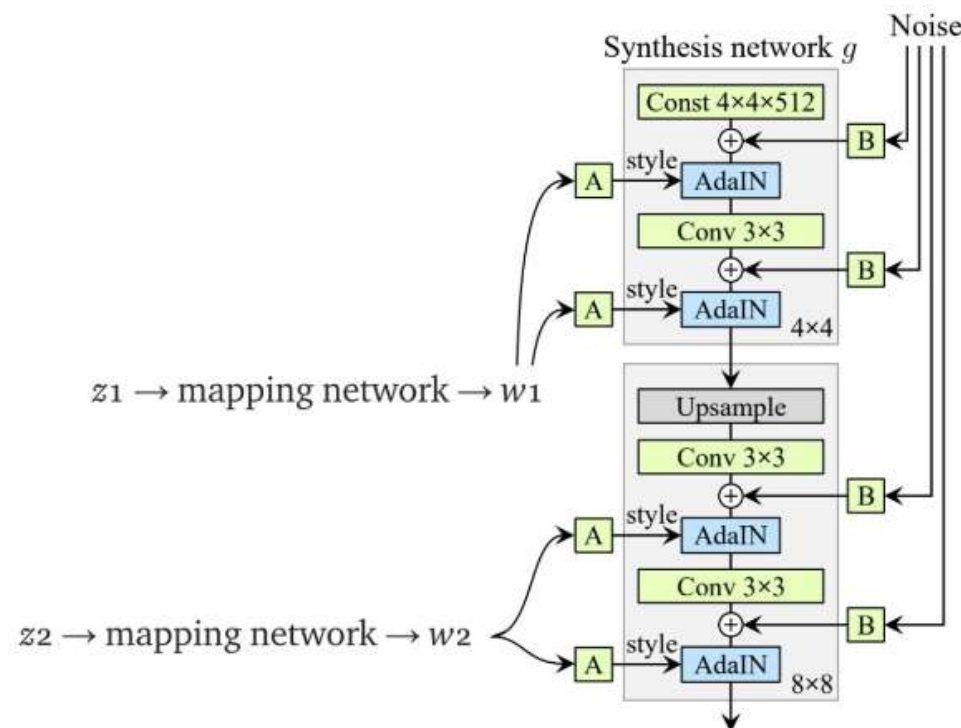
- Будем подавать шум в **Synthesis Network** на каждом уровне разрешения
- **Mapping Network** будет контролировать глобальный стиль (волосы, глаза)
- **Noise** будет отвечать за мелкие детали (морщины, веснушки)



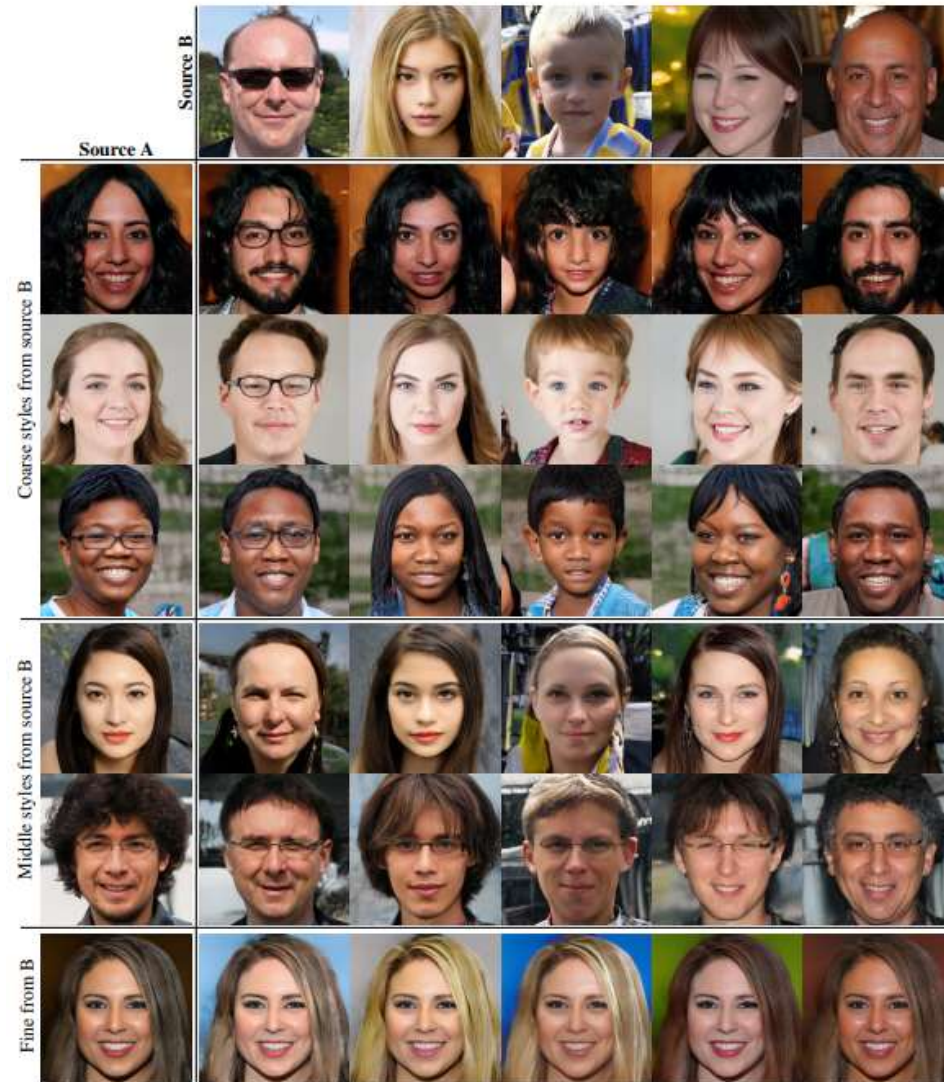
StyleGAN: Style Mixing

Можем попробовать взять 2 разных стилевых вектора w_1 и w_2

- На *coarse* слоях возьмем w_1 , а на *fine* – w_2



StyleGAN: Style Mixing



Спасибо за внимание!